



Instituto Tecnológico
de Buenos Aires

TPE
PROTOCOLOS DE COMUNICACIÓN

Segundo cuatrimestre de 2025

Grupo 11

Integrantes:

Francisco Mangiaterra	64544
Agostino Alfieri	64607
Nicolas Canzonieri	63501
Tomas Balboa	64237

Fecha de entrega: Jueves 11 de diciembre de 2025

Índice

1. Descripción detallada de los protocolos y aplicaciones desarrolladas.....	3
1.1. Arquitectura General del Proyecto.....	3
1.2. Protocolo SOCKS5 Handshake Inicial.....	3
1.3. Protocolo de Autenticación (RC 1929).....	4
1.4. Protocolo de Management.....	4
1.4.1. Comandos Disponibles.....	6
2. Problemas encontrados durante el diseño y la implementación.....	13
2.1 Problemas de compilación multiplataforma.....	13
2.2 Manejo de Memoria Dinámica y Recursos.....	13
2.3 getaddrinfo.....	14
3. Limitaciones de la aplicación.....	15
3.1. Gestión de Usuarios.....	15
3.2. Seguridad.....	15
3.3. Comandos Socks5.....	15
4. Posibles extensiones.....	15
5. Conclusiones.....	17
6. Ejemplos de prueba.....	17
7. Guía de instalación detallada y precisa.....	22
8. Instrucciones para la configuración.....	24
9. Ejemplos de configuración y monitoreo.....	26
10. Documento de diseño del proyecto.....	30

1. Descripción detallada de los protocolos y aplicaciones desarrolladas.

1.1. Arquitectura General del Proyecto

El proyecto consiste en la implementación de un servidor proxy SOCKS5, utilizando el lenguaje de programación C y siguiendo los lineamientos del RFC 1928. Emplea una arquitectura basada en un patrón de máquina de estados, utilizando un selector multiplexor y manejando múltiples conexiones concurrentes de manera no bloqueante. Por otro lado. Además del proxy en sí, el proyecto incorpora módulos de logging, métricas y un canal de administración propio que permiten observar y gestionar el servidor en tiempo real.

Una aclaración importante a tener en cuenta es que siempre que hablemos de “VERSION” nos estamos refiriendo a la versión del protocolo, que en resumen sería una forma de aclarar el “idioma” que el servidor y el cliente van a estar hablando. Es decir, es un número que indica que formato de mensajes se está usando.

1.2. Protocolo SOCKS5 Handshake Inicial

Este es el primer protocolo con el que interactúa el cliente. En resumen es un breve handshake, en el que el cliente informa que métodos de autenticación soporta y el servidor elige uno.

El formato de request (cliente → servidor) para la negociación de métodos es el siguiente:

VERSION	NMETHODS	METHODS
1 byte	1 byte	N bytes

- ❖ **Version:** 0x05
- ❖ **Nmethods:** La cantidad de métodos de autenticación que el cliente anuncia.
- ❖ **Methods:** Lista de métodos soportados por el cliente (entre 1 y 255 bits de largo).

El formato de respuesta (servidor → cliente):

VERSION	METHOD
---------	--------

1 byte	1 byte
--------	--------

- ❖ **Version:** 0X05
- ❖ **Method:** Método de autenticación elegido por el servidor entre los que ofreció el cliente.

1.3. Protocolo de Autenticación (RC 1929)

Una vez realizado el handshake, (si el servidor eligió el método de username/password) entra en juego nuestro protocolo de autenticación. Basado en el RFC 1929, el servidor exige que el cliente se identifique explícitamente enviando sus credenciales (usuario y contraseña). Estas credenciales se validan contra una lista de usuarios configurada en el servidor y solo si la misma es exitosa, es que se le permite al cliente acceder a las funcionalidades provistas por nuestro servidor.

El formato de request (cliente → servidor):

VERSION	USER LENGTH	USERNAME	PASSWORD LENGTH	PASSWORD
1 byte	1 byte	N bytes	1 byte	N bytes

La estructura del mensaje de autenticación es:

- ❖ **Version:** 0x01
- ❖ **User-Length:** Longitud del nombre de usuario (menor a 64).
- ❖ **Username:** El nombre de usuario (entre 1 y 64 bits de largo).
- ❖ **Password-Length:** Longitud de la contraseña (menor a 64).
- ❖ **Password:** La contraseña (entre 1 y 64 bits de largo).

El formato de respuesta (servidor → cliente):

VERSION	STATUS
1 byte	1 byte

- ❖ **Version:** 0x01
- ❖ **Status:** El resultado de la autenticación (0x00 = exitosa | 0xFF = fallida)

1.4. Protocolo de Management

Adentrándonos al protocolo de management, este protocolo actúa como una especie de “panel de control remoto” del servidor SOCKS5. La idea consiste en que en vez de tocar archivos de configuración, podamos conectarnos por TCP, autenticarnos y luego poder mandar comandos para administrar el servidor.

Esto incluye por ejemplo: gestionar usuarios (listar, crear, eliminar o ver historial de actividad), ver estadísticas del servidor, cambiar quien es administrador y quien no o también cambiar el método de autenticación por defecto del SOCKS (no auth o usuario-contraseña).

El flujo básico es el siguiente:

1. El cliente abre TCP al puerto de management.
2. Sucede la autenticación con username/password (RF 1929) (ya detallado)
3. En caso de una autenticación exitosa, el cliente envía un comando de management. El formato es el siguiente:

Request de comando (cliente → servidor):

VERSION	COMMAND	LENGTH-PAYLOAD	PAYLOAD
1 byte	1 byte	1 byte	N bytes

❖ **Version:** 0x01

❖ **Command:** Código del comando (enum interno).

- 0x00 add: agrega un usuario.
- 0x01 del: borra un usuario.
- 0x02 users: lista usuarios.
- 0x03 stats: imprime métricas.
- 0x04 chrol: cambia los permisos de un usuario.
- 0x05 sauth: define el método de autenticación por defecto.
- 0x06 gauth: devuelve el método de autenticación por defecto.
- 0x07 audit: imprime un registro de las acciones de un usuario.
- 0x08 chpwd: cambia la contraseña de un usuario.

❖ **Length-Payload:** La longitud total del payload.

❖ **Payload:** Los datos asociados al comando (Ej: ADD_USER tincho 1234 → PAYLOAD = “tincho:1234”).

4. El servidor responde con el siguiente formato de respuesta:

Response de comando (servidor → cliente):

VERSION	STATUS	BODY
1 byte	1 byte	N byte

- ❖ **Version:** 0x01
 - ❖ **Status:** Es el estado lógico del comando:
 - OK (0x00): Ejecutado correctamente.
 - FORBIDDEN (0x01): El usuario no tiene permisos para ejecutar ese comando.
 - INVALID VERSION (0x02): La versión del protocolo de management no es soportada.
 - INVALID COMMAND (0x03): El comando recibido no corresponde a ningún comando válido.
 - INVALID ARGS (0x04): Los argumentos son inválidos (cantidad o formato).
 - INVALID LENGTH (0x05): La longitud del payload no coincide o es inválida.
 - SERVER ERROR (0x06): Error del servidor.
 - ❖ **Body:** Es un texto adicional que da un mensaje sobre la respuesta obtenida (de confirmación o de error).
5. Finalmente se cierra la conexión (un comando por conexión).
 6. Si la autenticación falla:
 - a. El servidor responde rechazo y cierra la conexión.

Por otro lado, se decidió implementar 2 tipos de usuarios: usuarios con rol **admin** que pueden ejecutar todos los comandos (gestionar usuarios, cambiar configuración, ver actividad) y usuarios **no admin** que solo pueden ejecutar los comandos que no están marcados como “*admin only*”.

1.4.1. Comandos Disponibles

Continuando con el protocolo de management, aquí explicaremos un poco más sobre los comandos disponibles y su semántica.

Todos los comandos siguen la misma estructura **request/response**, donde los campos **VERSION**, **COMMAND**, **LENGTH-PAYLOAD**, **PAYLOAD**, **STATUS** y **BODY (o MESSAGE)** conservan las definiciones ya establecidas anteriormente. Por otro lado, cuando en la definición de un comando aparece indicado *admin, significa que dicho comando sólo puede ser ejecutado por usuarios con privilegios de administrador.

1.4.2. ADD USER (MGMT_ADD_USER) *admin

Tiene como objetivo crear un nuevo usuario en el servidor.

El request que podría enviar el cliente por ejemplo, es el siguiente :

VERSION	COMMAND	PAYLOAD LENGTH	PAYLOAD
0x01	0x00	0x0C	tincho:1234

Luego, la respuesta del servidor sería una de las siguientes, lógicamente dependiendo del caso en el que nos encontremos:

VERSION	STATUS	BODY
0x01	OK(0x00)	<i>“user added”</i>
	FORBIDDEN (0x01)	<i>“command requires admin privileges”</i>
	INVALID ARGS (0x04)	<i>“user already exists”</i>
	SERVER ERROR (0x06)	<i>“failed to add user”</i>

1.4.3. DELETE USER (MGMT_DELETE_USER) *admin

Eliminar un usuario existente.

El request por parte del cliente para eliminar un usuario podría ser:

VERSION	COMMAND	PAYLOAD LENGTH	PAYLOAD
0x01	0x01	0x06	tincho

La respuesta podría ser una de las siguientes:

VERSION	STATUS	BODY
0x01	OK(0x00)	<i>“user deleted”</i>
	FORBIDDEN (0x01)	<i>“command requires admin privileges”</i>
	INVALID ARGS (0x04)	<i>“expected 1 args” or “user not found”</i>
	SERVER ERROR (0x06)	<i>“failed to delete user”</i>

1.4.4. LIST USERS (MGMT LIST_USERS)

Listar los usuarios registrados en el servidor.

Para este comando, el request del cliente es sin payload:

VERSION	COMMAND	PAYLOAD LENGTH
0x01	0x02	0x00

Y la respuesta seria:

VERSION	STATUS	BODY
0x01	OK(0x0)	<i>“users count: .. (*list of users)”</i>
	INVALID ARGS (0x04)	*Sucedee si el cliente envió sin querer un argumento al solicitar la lista de usuarios.

1.4.5. STATS (MGMT_STATS)

Obtener estadísticas globales del servidor.

El cliente envía el siguiente request para obtener las estadísticas del servidor:

VERSION	COMMAND	PAYLOAD LENGTH
0x01	0x03	0x00

Y la respuesta del servidor es:

VERSION	STATUS	BODY
0x01	OK(0x0)	Se detallan las métricas obtenidas, como por ejemplo: <i>“Current connections: .. Total connections: ... Max connections: ... Bytes sent/recieved: ... / ... etc ”</i>
	INVALID ARGS (0x04)	*Sucedee si el cliente envió sin querer un argumento al solicitar la lista de usuarios.

1.4.6. CHANGE ROLE (MGMT_CHANGE_ROLE) *admin

Cambiar el rol de un usuario (entre admin y no admin)

En este caso por ejemplo, esto sería una request para convertir a “tincho” en admin:

VERSION	COMMAND	PAYLOAD LENGTH	PAYLOAD
0x01	0x04	0x0C	tincho:admin

Consecuentemente, la respuesta por parte del servidor podría ser una de las siguientes:

VERSION	STATUS	BODY
0x01	OK(0x0)	<i>“role changed”</i>
	FORBIDDEN (0x01)	<i>“command requires admin privileges”</i>
	INVALID ARGS (0x04)	<i>“expected 2 argos” o “user not found” o “role must be ‘admin’ or ‘user’ ”</i>
	SERVER ERROR (0x06)	<i>“failed”</i>

1.4.7. SET DEFAULT AUTH (MGMT_SET_DEFAULT_AUTH) *admin

Configurar el método de autenticación por defecto del servidor SOCKS5.

VERSION	COMMAND	PAYLOAD LENGTH	PAYLOAD
0x01	0x05	0x07 o 0x11	<i>“no_auth”</i> o <i>“username_password”</i>

Consecuentemente, la respuesta por parte del servidor podría ser:

VERSION	STATUS	BODY
0x01	OK(0x0)	<i>“ok”</i>
	FORBIDDEN (0x01)	<i>“command requires admin privileges”</i>
	INVALID ARGS (0x04)	<i>“method must be ‘no_auth’ or ‘username_password’”</i> o <i>“expected 1 arg”</i>
	SERVER ERROR (0x06)	<i>“failed”</i>

1.4.8. GET DEFAULT AUTH (MGMT_GET_DEFAULT_AUTH) *admin

Obtener el método de autenticación por defecto del servidor SOCKS5. Al ser solamente una consulta de la configuración actual, no hay columna de payload y, consecuentemente, el payload length es de 0.

VERSION	COMMAND	PAYLOAD LENGTH
0x01	0x06	0x00

Por otro lado, la respuesta puede ser una de las siguientes:

VERSION	STATUS	BODY
0x01	OK(0x00)	"Default auth method: no_auth" o "Default auth method: username_password"
	FORBIDDEN(0x01)	<i>"command requires admin privileges"</i>
	INVALID ARGS (0x04)	<i>"get_default_auth: expected 0 args"</i>

1.4.9. USER_ACTIVITY (MGMT_USER_ACTIVITY) *admin

Se recolecta el historial de actividad de un usuario. Volviendo al ejemplo anterior del usuario “tincho”, este sería un posible request:

VERSION	COMMAND	PAYLOAD LENGTH	PAYLOAD
0x01	0x07	0x06	tincho

La respuesta del servidor puede ser una de las siguientes:

VERSION	STATUS	BODY
0x01	OK(0x0)	En este caso se imprimirá en el body algo como: <i>“Access history for tincho (3 records):</i> - 2025-12-10 10:15:23 - management auth success - 2025-12-10 10:16:05 - 142.250.184.100:443 - 2025-12-10 10:20:11 - management auth failed ”
	FORBIDDEN (0x01)	<i>“command requires admin privileges”</i>
	INVALID ARGS (0x04)	<i>“expected 1 argument”</i>
	SERVER ERROR (0x06)	<i>“no activity logs found for user”</i>

1.4.10. CHANGE PASSWORD (MGMT_CHANGE_PASSWORD) *admin

El mensaje que se envía como request por parte del usuario para cambiar la contraseña, es el siguiente:

VERSION	COMMAND	PAYLOAD LENGTH	PAYLOAD
---------	---------	----------------	---------

0x01	0x08	0x0C	tincho:0000
------	------	------	-------------

Consecuentemente, la respuesta del servidor puede ser alguna de las siguientes:

VERSION	STATUS	BODY
0x01	OK(0x0)	<i>“password changed”</i>
	FORBIDDEN (0x01)	<i>“command requires admin privileges”</i>
	INVALID ARGS (0x04)	<i>“expected 2 args”</i> o <i>“user not found”</i>
	SERVER ERROR (0x06)	<i>“failed”</i>

Cerrando con esta sección, para la correcta implementación de este protocolo fue vital el uso de dos parsers separados: el parser de autenticación `auth_parser` (reutilizado de SOCKS5) y el parser específico de comandos `mgmt_command_parser`.

El primero interpreta el subprotocolo RFC 1929 ([VER] [ULEN] [USERNAME] [PLEN] [PASSWORD]), valida las credenciales vía `users_authenticate()` y marca si el cliente es admin; una vez autenticado, la conexión pasa a `mgmt_command_parser`, que lee mensajes [MGMT_VERSION][CMD][LEN][PAYLOAD], separa argumentos por : y dispara el handler correspondiente. De esta forma, `auth_parser` garantiza quién es el cliente y sus permisos, y `mgmt_command_parser` ejecuta la operación de administración, devolviendo siempre un código de estado y un mensaje coherente.

1.5. Servidor SOCKS5

Ya detallados todos los protocolos que forman parte del proyecto, es momento de desarrollar en mayor profundidad lo que hace nuestra aplicación. El servidor es un proxy SOCKS5 no bloqueante que acepta conexiones TCP entrantes de clientes y las reenvía a servidores de destino (en nombre del cliente). Utilizando una máquina de estados y un selector logra manejar múltiples conexiones/sockets simultáneamente de forma no bloqueante.

Cada conexión tiene asociada una máquina de estados (stm) la cual va avanzando por las distintas etapas de los protocolos mencionados anteriormente (handshake inicial, autenticación (RFC 1929), parseo del request y transferencia de datos).

La aplicación consecuentemente logra “proxyear” tráfico SOCKS5 real y manejar varias conexiones en simultáneo. Por otro lado, se puede controlar quién puede usar el proxy, ya que hay un sistema de usuarios interno.

Por lo tanto, solo las credenciales válidas pueden pasar la etapa de autenticación y usar el servicio (o utilizarlo sin autenticación). Finalmente, la aplicación también permite consultar estadísticas del servidor y logear actividad de usuarios. Cada vez que un usuario usa el proxy se registran esos accesos en una estructura de logs que luego puede ser consultada desde el canal de management.

1.6. Cliente

Sacando la implementación del servidor, también se armó un cliente específico de administración en C que permite conectarse al puerto de management del proxy, autenticarse con un usuario y contraseña válidos y posteriormente ejecutar los distintos comandos del protocolo de management.

Consecuentemente, gracias a esta implementación, un administrador (o un usuario sin privilegios), puede interactuar con el servidor. Sin la necesidad de tener que construir a mano los mensajes binarios, ya que simplemente debe invocar el binario con sus credenciales y el comando deseado.

El cliente ofrece una interfaz de línea de comandos simple, en donde se indica: host, puerto, username, password y luego el comando de administración deseado (cualquiera de los mencionados anteriormente). Además, el usuario puede usar “def def” y utilizar el host y puerto definidos por default (1080, 127.0.0.1), en el caso que desee no definirlos.

Internamente, el cliente se ocupa de las tareas vinculadas con abrir el socket, construir y mandar el mensaje de autenticación (RFC 1929), serializar el mensaje de management, etc. Gracias a esto, el usuario solo ve mensajes claros en textos, mientras que los detalles del protocolo y el manejo de sockets ocurre por detrás.

2. Problemas encontrados durante el diseño y la implementación.

2.1 Problemas de compilación multiplataforma

Durante el desarrollo, surgieron problemas de portabilidad entre sistemas operativos. Dos de los integrantes tenemos Windows, y los dos restantes, MacOs. En reiteradas ocasiones, cuando pusheaban cambios los de MacOs, el código no compilaba para nosotros, los de Windows, y viceversa. Si bien eran intermitentes y difíciles de predecir, ninguno presentaba una dificultad real de resolución ya que solían ser en su mayoría warnings, que

debido al -Werror en el Makefile hacían que falle la compilación. Sin embargo, pese a su baja dificultad, era muy molesto y al comienzo generaba altercados entre los integrantes porque parecía que se pusheaban commits que no compilaban.

Intentamos identificar si estaba relacionado con diferencias en versiones de gcc entre plataforma o variaciones en las implementaciones de la biblioteca estándar C, o de caracteres fin de línea (CRLF VS LF), aunque no dimos con la causa raíz. Pese a que era molesto y ralentizaba un poco el flujo de trabajo, lo resolvimos en cada instancia con ajustes menores y recompilación (make clean & make all). Al no afectar la funcionalidad final del servidor, y teniendo en cuenta que el desarrollo del proyecto fue contrarreloj, no invertimos más recursos en investigarlo exhaustivamente. Pensamos en un futuro que con un contenedor de Docker para garantizar consistencia de compilación o configurar CI/CD podríamos solucionarlo.

2.2 Manejo de Memoria Dinámica y Recursos

El manejo de memoria dinámica fue crítico, sobre todo al tener que gestionar buffers de tamaño variable, estructuras de usuarios, y resultados de resoluciones DNS. Además, como cada conexión (cliente y destino) abre file descriptors que deben cerrarse correctamente en todos los caminos de error para evitar leaks de recursos del sistema

Se implementaron rutinas de limpieza cuidadosas en socks5_destroy() (provisto por la cátedra) que liberan memoria de buffers, estructuras addrinfo de resoluciones DNS, y cierran todos los file descriptors asociados. Se agregaron conteos de referencias para evitar doble liberación (lo cual nos llevó un tiempo considerable comprender). En request_handler.c, se asegura que si getaddrinfo() fue exitoso, siempre se llame a freeaddrinfo() antes de destruir la conexión.

2.3 getaddrinfo

Al ser getaddrinfo() una función bloqueante que puede tardar varios segundos en resolver nombres de dominio (y casi siempre efectivamente los tarda), especialmente en casos de timeout DNS, utilizamos getaddrinfo_a() (asíncrona). No exento de problemas nos encontramos con que: requiere libanl en Linux, no está disponible en macOS, y su API es compleja.

Claramente ejecutar getaddrinfo() en el hilo principal iba a bloquear todas las demás conexiones activas, por lo que implementamos un pool de threads workers (resolver_pool.c) con 10 threads que toman trabajos de una cola protegida por mutex.

Los threads ejecutan getaddrinfo() de forma bloqueante (pero en background), y notifican al selector principal mediante un pipe cuando la resolución está completa. El selector detecta el evento de lectura en el pipe y continúa la FSM. Este diseño híbrido evita bloquear el hilo principal mientras mantiene simplicidad sobre getaddrinfo_a(). Se agregó timeout de 5 segundos en la cola para evitar deadlocks cuando está llena.

2.4 Manejo de Resolución Múltiple

Cuando un FQDN resuelve a múltiples IPs (como por ejemplo google.com que tiene alrededor de 10 IPs distintas), si la primera falla, teníamos que resolver como intentar con las siguientes sin bloquear. Para eso, hicimos que `client_info` mantenga `origin_resolution` (lista completa de `addrinfo`) y `current_resolution` (puntero al intento actual). En el estado `CONNECTING`, si `connect()` falla (no inmediatamente exitoso), se avanza `current_resolution = current_resolution->ai_next` y se reintenta. Esto básicamente nos permitió un failover automático entre múltiples IPs.

2.5 Métricas de bytes en cero

Cuando observábamos las stats con `client bytes_sent` y `bytes_received` siempre mostraban 0 a pesar de descargas exitosas de más de 10mb. Luego de buscar un tanto dimos con la causa raíz y era que en `copy.c`, `metrics_update()` se llamaba con lógica invertida (`is_client ? bytes_read : bytes_write`) y faltaban llamadas después de `recv()`. Lo que hicimos entonces fue agregar una `read_aux()` y `write_aux()` incluyendo en estas `metrics_update(0, bytes_written)` para cada caso de si es cliente o no y después de `recv()` (en `read_aux`)

3. Limitaciones de la aplicación.

3.1. Gestión de Usuarios

Tenemos un sistema de gestión de usuarios básico, los cuales, valga aclarar, no se encuentran hasheados, sino que en todo momento sus credenciales viven en RAM. Esto tiene la consecuencia directa de hacer que cualquier reinicio del servidor elimina toda la información de usuarios creados o modificados durante la sesión, dificultando el análisis histórico del rendimiento y uso del sistema. De esta manera, no existe la persistencia de configuración ni de estadísticas. El único usuario que vamos a poder utilizar en las diferentes sesiones es el usuario admin. Lo que nos lleva a la limitación de permitir una sola conexión administrativa a la vez.

3.2. Seguridad

Como mencionamos en el punto 3.1, la ausencia de cifrado en las comunicaciones (pues tanto usuarios como contraseñas se transmiten en texto plano), sumado al hecho de no tener una blacklist/whitelist, para controlar destinos permitidos o bloqueados, restringe significativamente su uso en entorno de alta seguridad, haciéndolo más adecuado para ambientes de desarrollo, pruebas o educativos donde estos aspectos no sean críticos.

3.3. Comandos Socks5

La implementación de nuestro proyecto cumple completamente con el comando CONNECT, pero no ha sido implementado tanto el comando BIND (referido a conexiones entrantes del destino) como el UDP ASSOCIATE (tunelización de datagramas UDP y su respectivo tráfico). Si bien estaban fuera del alcance de los requerimientos pedidos y las conexiones TCP salientes representan el caso de uso más común, sin lugar a dudas implementar ambos comandos sería un desafío interesante para llevar a cabo.

4. Posibles extensiones.

4.1. Soporte TLS/SSL

Agregar cifrado de la comunicación entre cliente y proxy mediante OpenSSL. La arquitectura actual separa claramente las operaciones de I/O (`copy.c`, `auth.c`), permitiendo reemplazar las llamadas `recv()`/`send()` por `SSL_read()`/`SSL_write()` sin afectar la lógica de la máquina de estados. El selector basado en `epoll`/`select` soporta nativamente sockets SSL en modo no bloqueante.

4.2. Rate Limiting

Implementar limitación de tasa por IP de origen o usuario para mitigar ataques de denegación de servicio. El punto de integración natural es durante el handshake (`hello_read`), antes de asignar recursos del servidor. El sistema de métricas existente (`metrics.c`) ya recolecta conexiones por segundo, facilitando la implementación.

4.3. Filtrado de Destinos

Agregar blacklist/whitelist de dominios/IPs destino para control de acceso granular. La verificación debe ocurrir en `request_handler.c`, inmediatamente después del parseo de la petición SOCKS5 pero antes de la resolución DNS, evitando consumir recursos del pool de resolución para destinos bloqueados.

4.4. Completar el Estándar RFC 1928

Comando BIND y UDP ASSOCIATE

Permitir que clientes soliciten que el proxy escuche conexiones entrantes en su nombre.

Soportar relay de paquetes UDP para aplicaciones como DNS o streaming.

4.5. Sniffing en POP3

La consigna opcional, solo obligatoria para la segunda entrega, que propone monitorear y registrar las credenciales de acceso simil ettercap.

4.6. Base de datos

En un futuro sería de gran utilidad crear una base de datos que nos permita persistencia de usuarios entre distintas conexiones del servidor SOCKS5, eliminado de esta manera el usuario default de administrador y que se pueda ingresar con un usuario propio al utilizar el servidor. Según lo que investigamos, SQLite sería una gran opción dado que tiene una librería estándar, funciona perfecto para aplicaciones embebidas y no tiene un servidor sino que es un archivo único

5. Conclusiones

Desarrollar este trabajo no fue fácil. Quizás la parte más complicada fue comenzar por la variedad de implementaciones que se debían realizar y por la interdependencia de ellas. Antes de tomar una decisión, se empezó por integrar los parches de la cátedra y leer los RFCs 1928 y 1929. Finalmente, se pudo programar una entrega con la que nos sintamos satisfechos.

Por capricho nuestro, se decidió implementar también un sistema de logging para así tener archivos donde se registren cada ejecución. Otros detalles que decidimos agregar fue un registro de consultas DNS realizadas y tiempo de vida del servidor, ambas se imprimirían con el resto de métricas solicitadas por la consigna.

La interdependencia e interconectividad de varios de los componentes de este proyecto fue en nuestra opinión lo más interesante del proyecto. En especial, conectar los comandos del cliente con los comandos de management del servidor.

Nuevamente, estamos satisfechos con nuestro trabajo y hacer un servidor socks5 fue una experiencia que agradecemos haber tenido. Nos hubiera gustado poder implementar la consigna de la segunda entrega, pero por manejos de tiempo no pudo ser posible. Aún así, poder poner en práctica gran parte de lo visto a lo largo de la materia fue enriquecedor y algo que no dudamos nos será muy útil en el futuro.

6. Ejemplos de prueba.

6.1. Conectividad con autenticación

Comando	<code>curl -x socks5://admin:0000@127.0.0.1:1080 http://ipinfo.io/ip</code>
Resultado esperado:	<code><tu_ip></code>

6.2. Conexión sin autenticación (NO_AUTH)

Comando	<code>curl -x socks5://127.0.0.1:1080 http://google.com -I</code>
Resultado esperado:	<code>HTTP/1.1 301 Moved Permanently Location: http://www.google.com/ Content-Type: text/html; charset=UTF-8</code>

6.3. Stress test - 500 conexiones concurrentes

Comando	<code>python3 tools/stress_clients.py 500 127.0.0.1 1080 60</code>
Resultado esperado:	<code>0 reply 0502 1 reply 0502 2 reply 0502 ... 499 reply 0502</code>

6.4. Estadísticas del servidor

Comando	<code>./bin/client 127.0.0.1 8080 admin 0000 stats</code>
Resultado esperado:	<code>+OK: stats Current connections: 0 Total connections: 523 Max connections: 500 Bytes sent: 15728640</code>

	Bytes received: 52428800 DNS queries: 156 Uptime (s): 3600
--	--

6.5. Monitoreo en tiempo real

Comando	<code>while true; do clear; date; echo ""; ./bin/client 127.0.0.1 8080 admin 0000 stats; sleep 1; done</code>
Resultado esperado:	(Stats actualizándose cada segundo)

6.6. Listado de usuarios

Comando	<code>./bin/client 127.0.0.1 8080 admin 0000 users</code>
Resultado esperado:	+OK: users admin (admin) user (user)

6.7. Auditoría de usuario

Comando	<code>./bin/client 127.0.0.1 8080 audit admin</code>
Resultado esperado:	+OK: audit 2025-12-10T12:15:23 - socks5 auth success 2025-12-10T12:15:24 - google.com:443 2025-12-10T12:20:15 - example.org:80

6.8. Logs en tiempo real

Comando	<code>tail -f \$(ls -t ./log/*.log grep -v test.log head -1)</code>
Resultado esperado:	<pre>2025-12-10T12:15:23[INFO] [HELLO] selected method=2 for fd=5 2025-12-10T12:15:23[INFO] [AUTH] fd=5 User 'admin' Authenticated (admin) 2025-12-10T12:15:23[DEBUG] [REQUEST] request_init fd=5</pre>

6.9. Filtrado de errores y warnings

Comando	<code>tail -f \$(ls -t ./log/*.log grep -v test.log head -1) grep --line-buffered -E '\[ERROR\] \[WARNING\]'</code>
Resultado esperado:	<pre>2025-12-10T13:45:45[WARNING] [USER_VALIDATION] Autenticacion fallida para usuario 'rocky' 2025-12-10T13:45:45[WARNING] [AUTH] fd=7 User 'rocky' Authentication failed</pre>

6.10. Estadísticas del log

Comando	<code>LOG=\$(ls -t ./log/*.log grep -v test.log head -1)</code>
Resultado esperado:	<pre>echo "Total lines: \$(wc -l < \$LOG)" echo "INFO: \$(grep -c '\[INFO\]' \$LOG)" echo "WARNING: \$(grep -c '\[WARNING\]' \$LOG)" echo "ERROR: \$(grep -c '\[ERROR\]' \$LOG)" Resultado esperado: Total lines: 1451 INFO: 520 WARNING: 210 ERROR: 0</pre>

6.11. Búsqueda de actividad de usuario

Comando	<code>grep "User 'admin'" \$(ls -t ./log/*.log grep -v test.log head -1)</code>
Resultado esperado:	<code>2025-12-10T12:15:23[INFO] [AUTH] fd=5 User 'admin' Authenticated (admin) 2025-12-10T12:20:45[INFO] [AUTH] fd=8 User 'admin' Authenticated (admin)</code>

6.12. Test completo de autenticación (Python)

Comando	<code>python3 tools/test_auth.py localhost 1080 admin 0000</code>
Resultado esperado:	<code>=== Test 1: NO credentials (NO_AUTH) === [✓] Server selected: NO_AUTH [✓] Connection SUCCESS === Test 2: Valid credentials (admin:0000) === [✓] Server selected: USER_PASS [✓] Auth SUCCESS [✓] Connection SUCCESS === Test 3: Invalid credentials === [✓] Server selected: USER_PASS [X] Auth FAILED (status=255)</code>

6.13. Autenticación exitosa con curl

Comando	<code>curl -x socks5://admin:0000@127.0.0.1:1080 http://example.org -I</code>
---------	---

Resultado esperado:	HTTP/1.1 200 OK Accept-Ranges: bytes Content-Type: text/html Content-Length: 1256
----------------------------	--

6.14. Autenticación fallida con curl

Comando	<pre>curl -x socks5://admin:wrongpass@127.0.0.1:1080 http://example.org</pre>
Resultado esperado:	<pre>curl: (97) User was rejected by the SOCKS5 server (1 255).</pre>

6.15. Test SOCKS5 completo - IPv4 y dominio

Comando	<pre>python3 tools/test_socks5.py admin 0000</pre>
Resultado esperado:	<pre>=== Test 1: IPv4 (ATYP=0x01) === [OK] SOCKS5 HELLO: auth method username/password [OK] Auth successful for user 'admin' [OK] Connected to 93.184.216.34:80 [SUCCESS] IP connection working! === Test 2: Domain name === [OK] SOCKS5 HELLO: auth method username/password [OK] Auth successful for user 'admin' [OK] Connected to example.org:80 [SUCCESS] SOCKS5 proxy working with domain!</pre>

6.16. Test de throughput

Comando	<pre>time curl -x socks5://admin:0000@127.0.0.1:1080 http://ipv4.download.thinkbroadband.com/10MB. zip -o /tmp/test.zip</pre>
---------	---

	<pre>./bin/client 127.0.0.1 8080 admin 0000 stats grep -E "Bytes sent Bytes received"</pre>
Resultado esperado:	<pre>real 0m2.345s user 0m0.123s sys 0m0.045s Bytes sent: 10485760 Bytes received: 2048</pre>

7. Guía de instalación detallada y precisa.

Para la compilación y ejecución del programa se necesita obtener código fuente. Para eso hay dos opciones.

1. Clonar desde repositorio Git:

```
git clone https://github.com/TomasBalboa/TP_PROTOS.git
cd TP_PROTOS
```

2. Descomprimir archivo .tar.gz o .zip

```
tar -xzf TPE-PROTOS.tar.gz
cd TPE_PROTOS
```

- a. Para zip:

```
unzip TPE-PROTOS.zip
cd TPE-TROPOS
```

Para la compilación es necesario tener GCC y Make en sus versiones más recientes. El proyecto utiliza un Makefile que automatiza todo el proceso de compilación.

```
make all
```

Este comando compila todos los archivos fuente (.c) a objetos (.o) y genera el ejecutable del servidor en bin/socks5d.

También es posible compilar el proyecto modo Debug

```
make all BUILD=debug
```

De esta forma se incluyen los símbolos de debugging (-g), se habilitan todos los logs incluyendo LOG_DEBUG, y no optimiza el código para facilitar el debugging.

Para la compilación del cliente, si bien puede hacerse conjuntamente a make all:

```
make all client
```

también podemos compilar el Cliente de Management por separado:

`make client`

Para limpiar compilación previa:

`make clean`

Elimina todos los archivos objeto y binarios generados y se limpia el proyecto antes de distribuirlo.

El servidor SOCKS5 se corre desde TPE-PROTOS (la raíz del proyecto) de la siguiente manera:

`./bin/socks5d (args)`

Para ver los distintos comandos y opciones disponibles se puede correr lo siguiente

`./bin/socks5d -h`

Vale aclarar que el logging puede fallar al ejecutar el binario. En tal caso, se procede a desplazar a la carpeta de log y dar permisos a todos los archivos:

`cd log`

`chmod 777 *.log`

Por último, para compilar el cliente se corre, también desde la raíz del proyecto. En lugar de host y port se puede especificar “def” para utilizar el puerto y host por defecto, 1080 y 127.0.0.1 respectivamente:

`./bin/client <host> <port> <username> <password> comando`

8. Instrucciones para la configuración.

Al iniciar el servidor SOCKS5 por primera vez, este crea automáticamente un usuario administrador con credenciales por defecto. Este usuario tiene permisos completos para gestionar el sistema a través del protocolo de management. Las credenciales del Administrador por Defecto:

Usuario: admin

Contraseña: 0000

El servidor permite configurar las direcciones y puertos donde va a escuchar el servicio SOCKS5 como el servicio de management mediante flags de línea de comandos.

Las opciones disponibles luego de `./bin/socks5d` son

-h

- Muestra la ayuda con todas las opciones disponibles y termina.

-l <SOCKS_ADDR>

- Dirección IP donde escuchará el servidor SOCKS5. Por defecto: 0.0.0.0 (todas las interfaces). Ejemplos:
 - **-l 0.0.0.0** # Escuchar en todas las interfaces (acceso remoto)
 - **-l 127.0.0.1** # Solo localhost (acceso local)
 - **-l 192.168.1.100** # IP específica de la máquina

-p <SOCKS_PORT>

- Puerto TCP donde escuchará el servidor SOCKS5. Por defecto: 1080
 - Rango válido: 1-65535
 - Ejemplo: **-p 9090**

-L <CONF_ADDR>

- Dirección IP donde escuchará el servidor de management.
 - Por defecto: **127.0.0.1** (solo localhost por seguridad)
 - Ejemplo: **-L 127.0.0.1**

-P <CONF_PORT>

- Puerto TCP donde escuchará el servidor de management.
 - Por defecto: **8080**
 - Ejemplo: **-P 8888**

-u <USERNAME>:<PASSWORD>

- Crea un usuario adicional al iniciar el servidor. Puede repetirse hasta 10 veces para crear múltiples usuarios. Los usuarios creados con **-u** NO tienen permisos de administrador.
 - Formato: **<nombre:contraseña>**
 - Restricciones:
 - Username: máximo 64 caracteres
 - Password: máximo 64 caracteres
 - Ejemplos:
 - **-u juan:password123**
 - **-u maria:clave456**

-V

- Muestra información sobre la versión del servidor y termina.

Hacemos una muestra de ejemplo con -l, -p, -P, -u:

```
fmangiaterra@LAPTOP-TF4BVUK5:~/TPE-PROTOS$ ./bin/socks5d -l 172.30.215.76 -p 9090 -P 8888 -u Sisifo:09122018
2025-12-11T00:42:16 Escuchando puerto TCP 9090 en 172.30.215.76
2025-12-11T00:42:16 Management server listening on 127.0.0.1:8888
2025-12-11T00:42:16 Default admin user: admin / 0000
2025-12-11T00:42:16 Additional users created: 1
```

Veamos que se creó el usuario:

```
fmangiaterra@LAPTOP-TF4BVUK5:~/TPE-PROTOS$ ./bin/client def def admin 0000 users
+OK: users
users count: 2
admin
Sisifo
```

Vemos que si utilizamos las credenciales correctas podemos utilizar el proxy con autorización. En este caso solo podemos usar el usuario admin, o Sisifo recientemente creado.

```
fmangiaterra@LAPTOP-TF4BVUK5:~/TPE-PROTOS$ curl --socks5 172.30.215.76:9090 --proxy-user Sisifo:09122018 http://example.org
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:
#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style><body><div><h1>Examp
le Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.<p><a href="https://iana.org/domains/ex
ample">Learn more</a></div></body></html>
```

Si no mandamos ninguna credencial también se puede utilizar sin autorización:

```
fmangiaterra@LAPTOP-TF4BVUK5:~/TPE-PROTOS$ curl --socks5 172.30.215.76:9090 http://example.org
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:
#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style><body><div><h1>Examp
le Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.<p><a href="https://iana.org/domains/ex
ample">Learn more</a></div></body></html>
```

Ahora veamos, con otra sintaxis, qué sucede si mandamos credenciales erróneas.

```
fmangiaterra@LAPTOP-TF4BVUK5:~/TPE-PROTOS$ curl -x socks5://Sisifo:20062007@172.30.215.76:9090 http://example.org
curl: (97) User was rejected by the SOCKS5 server (1 255).
```

Y si vamos al logueo vemos que se rechazó al usuario por contraseña incorrecta

```
2025-12-11T00:50:09[INFO] [AUTH] auth_process fd=5 user='Sisifo'
2025-12-11T00:50:09[WARNING] [USER_VALIDATION] Autenticacion fallida para usuario 'Sisifo'
2025-12-11T00:50:09[WARNING] [AUTH] fd=5 User 'Sisifo' Authentication failed
2025-12-11T00:50:09[DEBUG] [AUTH] auth_read_close fd=5
2025-12-11T00:50:09[DEBUG] [AUTH] auth_write fd=5 wrote=2 bytes
2025-12-11T00:50:09[INFO] [AUTH] fd=5 Auth failed, closing connection after sending response
```

9. Ejemplos de configuración y monitoreo.

9.1. Cliente de Monitoreo:

9.1.1. Conexión al servidor

```

tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000
Usage: ./bin/client <host> <port> <username> <password> <command> [args]
You can use 'default' for host or port to use default values (localhost and 8080)

Available commands:
HELP                                - Show all commands
LIST_USERS                          - List all users
STATS                               - Show server statistics
ADD_USER <username> <password>      (admin only) - Register a new user
DELETE_USER <username>               (admin only) - Delete a user
CHANGE_PASSWORD <username> <new_password> (admin only) - Change user password
CHANGE_ROLE <username> <admin|user>    (admin only) - Change user role
AUDIT_USER <username>                (admin only) - Show user activity log
GET_DEFAULT_AUTH                     - Get current auth method
SET_DEFAULT_AUTH <no_auth|username_password> (admin only) - Set default auth method

Command aliases for convenience:
help, users, stats, add, del, chpwd, chrol, audit, gauth, sauth

```

Imagen 1: Intentando conectar con el cliente usando todo por default.

Aunque se dejó vacío el argumento vinculado al comando, haciendo que el input sea inválido, igualmente se despliegan todos los comandos disponibles.

```

tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 help
Usage: <exe> <host> <port> <username> <password> <command> [args]
Use 'default' as host or port if you wish to use the default values (localhost and 8080)

help
    prints existing commands (-h is an alias)
users
    prints existing users
stats
    prints server stats
add UNAME PWD (admin only)
    registers a new user into database
del UNAME (admin only)
    deletes a user from database
chpwd UNAME NEWPWD (admin only)
    changes a user's password
chrol UNAME (admin|user) (admin only)
    switches a user's admin status
audit UNAME (admin only)
    prints a user's activity log
gauth
    gets current default authentication method
sauth (no_auth|username_password) (admin only)
    sets default authentication method as no-auth or u/p

```

Imagen 2: Utilizando “help” se despliegan mejor los comandos.

Como se observa en la imagen 2, cuando se utiliza el comando help, los comandos se imprimen de forma más clara y se da una explicación más clara de que hace cada uno.

9.1.2. Add User

```

tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 add tomi 1234
+OK: user added

```

Imagen 3: Agregando un usuario con el comando add.

Como se observa, si utilizo las credenciales correctas de admin (admin 0000), puedo agregar un usuario a la lista sin problemas.

Sin embargo, veamos que si pongo mal las credenciales del administrador, tendremos una falla de autenticación:

```
⊗ tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 1111 add tomi 1234
Username and/or password may not be correct
Error: Authentication failed for user 'admin'
```

Imagen 4: Fallo de autenticación al agregar un usuario con credenciales incorrectas.

Esto sucede con cualquier otro comando que requiera permisos de administrador.

Por otro lado, si quiero agregar un usuario con el mismo username que el que acabo de agregar:

```
⊗ tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 add tomi 1234
-ERR: add user: user already exists
```

Imagen 5: Fallo de intento de agregar 2 veces el mismo usuario.

9.1.3. List Users

```
● tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 users
+OK: users
users count: 2
admin
tomi
```

Imagen 6: Despliegue de todos los usuarios cargados en el sistema.

Como se observa en la imagen 5, además del usuario que ya viene default del servidor, también se despliega el usuario recién creado “tomi”.

9.1.4. Delete User

```
Error: Authentication failed for user 'admin'
● tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 del tomi
+OK: user deleted
● tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 users
+OK: users
users count: 1
admin
```

Imagen 7: Muestra la eliminación de un usuario y como luego no se despliega en la lista de usuarios.

El usuario se elimina correctamente de la lista de usuarios, ya que luego ponemos el comando de list users y ya no figura en la lista.

Si intento eliminar un usuario que no existe:

```
⊗ tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 del tomi
-ERR: delete_user: user not found
```

Imagen 8: Intento fallido de eliminar un usuario inexistente.

9.1.5. Stats

```

tomas@MacBook-Air-Tomas TP_PROTO5 % ./bin/client def def admin 0000 stats
+OK: stats
Current connections: 0
Total connections: 0
Max connections: 0
Bytes sent: 0
Bytes received: 0
DNS queries: 0
Uptime (s): 2659

```

Imagen 9: Despliega algunas estadísticas del servidor.

9.1.6. Change Password

Si se desea cambiar la contraseña, se arroja el siguiente comando:

```

tomas@MacBook-Air-Tomas TP_PROTO5 % ./bin/client def def admin 0000 chpwd admin 1234
+OK: password changed
tomas@MacBook-Air-Tomas TP_PROTO5 % ./bin/client def def admin 0000 add pruebita 1111
Username and/or password may not be correct
Error: Authentication failed for user 'admin'

```

Imagen 10: Exhibe el cambio de contraseña.

Como se observa en la imagen 10, con el comando chpwd, se logró cambiar la contraseña correctamente. Ya que posteriormente - a manera de prueba- intentamos agregar un usuario usando las credenciales viejas y el sistema no lo permitió.

Por otro lado, si se le intenta cambiar la contraseña a un usuario inexistente:

```

tomas@MacBook-Air-Tomas TP_PROTO5 % ./bin/client def def admin 1234 chpwd pablito 1234
-ERR: change_password: user not found

```

Imagen 11: Muestra un intento de cambio de contraseña con usuario inexistente.

El sistema no lo permite y arroja que el usuario no fue encontrado.

9.1.7. Change Role

Si intento cambiar el rol de mi usuario, siendo un usuario no administrador:

```

tomas@MacBook-Air-Tomas TP_PROTO5 % ./bin/client def def tomi 1234 chrol tomi admin
-ERR: command requires admin privileges

```

Imagen 11: Intentando cambiar el rol desde un usuario normal.

Como se observa en la imagen 10, el sistema restringe cambiar el rol si el usuario no es administrador. Veamos ahora el otro caso:

```

tomas@MacBook-Air-Tomas TP_PROTO5 % ./bin/client def def admin 0000 chrol tomi admin
+OK: role changed

```

Imagen 12: Rol cambiado correctamente.

En este caso sí se logró cambiar correctamente el rol del usuario (usando las credenciales de un usuario administrador).

9.1.8. Get Default Auth Method

Para obtener el método de autenticación default del usuario, se ejecuta el siguiente comando:

```
● tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 gauth
+OK: default auth method
Default auth method: username_password
```

Imagen 13: Obtengo el método de autenticación default del usuario.

9.1.9. Set Default Auth Method

```
● tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 sauth no_auth
+OK: default auth method set
● tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 gauth
+OK: default auth method
Default auth method: no_auth
```

Imagen 14: Seteo el método de autenticación default del usuario.

Como se observa en la imagen 13, cambie el método de autenticación default a “no_auth” y cuando tiro el comando de “gauth” aparece cambiado correctamente.

9.1.10. Audit User

Nos da una auditoría de un usuario:

```
● tomas@MacBook-Air-Tomas TP_PROTOS % ./bin/client def def admin 0000 audit admin
+OK: activity log for 'admin'
Access history for admin (16 records):
2025-12-11 00:18:08 - management auth failed
2025-12-11 00:18:25 - management auth success
2025-12-11 00:18:33 - management auth success
2025-12-11 00:27:17 - management auth success
2025-12-11 00:28:57 - management %
```

Imagen 15: Auditoría para el usuario “admin”.

La imagen 14, refleja los distintos momentos de una autenticación exitosa o fallida que registró el sistema, utilizando el usuario “admin”.

Por otro lado, en este caso no aparece porque no se realizó ninguna, pero el sistema también registra conexiones SOCKS que el usuario auditado haya iniciado.

10. Documento de diseño del proyecto.

10.1. Arquitectura No Bloqueante

Todos los sockets operan en modo “O_NONBLOCK”.

Multiplexado I/O

Un único hilo maneja múltiples conexiones simultáneas mediante “select(2)”.

Thread Pool para DNS

Resoluciones DNS bloqueantes se delegan a un pool de threads trabajadores.

Máquinas de Estados

Cada conexión SOCKS5 y de management se modela como una máquina de estados finita.

Escalabilidad

Soporte para 500+ conexiones concurrentes.

10.2. Pool de Threads

Se implementó un work crew siguiendo el patrón descrito por Butenhof en "Programming with POSIX Threads" (Sección 4.2). Este patrón consiste en:

El Resolver Pool implementa un pool de threads trabajadores para DNS:

Job Queue (buffer circular)

↓

[Worker 1] [Worker 2] ... [Worker N]

↓

getaddrinfo() (bloqueante)

↓

selector_notify_block() → wakeup main thread

Archivo: `src/resolver\_pool.c`

Problema: `getaddrinfo()` es bloqueante y detendría todo el servidor.

Solución: Pool de threads trabajadores que ejecutan DNS en paralelo.

10.3. Parsers

Hello Parser (`hello\_parser.c`)

VER	NMETHODS	METHODS
0x05	1B	1 a 255 bits

Auth Parser (`auth\_parser.c`)

VER	ULEN	UNAME	PLEN	PASSWD
0x01	1B	1 a 64 bits	1B	1 a 64 bits

Request Parser (`request\_parser.c`)

VER	CMD	RSV	ATYP	DST.ADDR	DST.PORT
0x05	1B	0x00	1B	Variable	2B

ATYP:

- 0x01: IPv4 (4 bytes en DST.ADDR)
- 0x03: FQDN (1 byte length + domain en DST.ADDR)
- 0x04: IPv6 (16 bytes en DST.ADDR)

10.4. Diagramas

10.4.1. Estados socks5

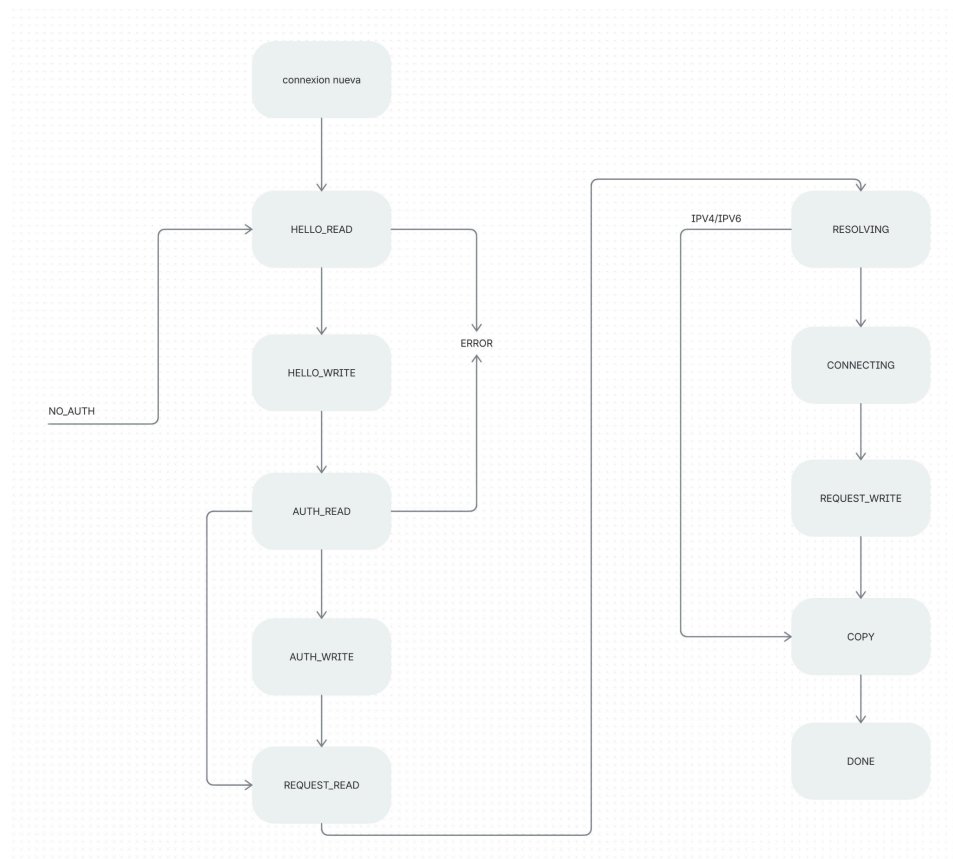


Diagrama 1: Refleja el flujo del servidor a partir de una conexión nueva.

10.4.2. Flujo de Datos

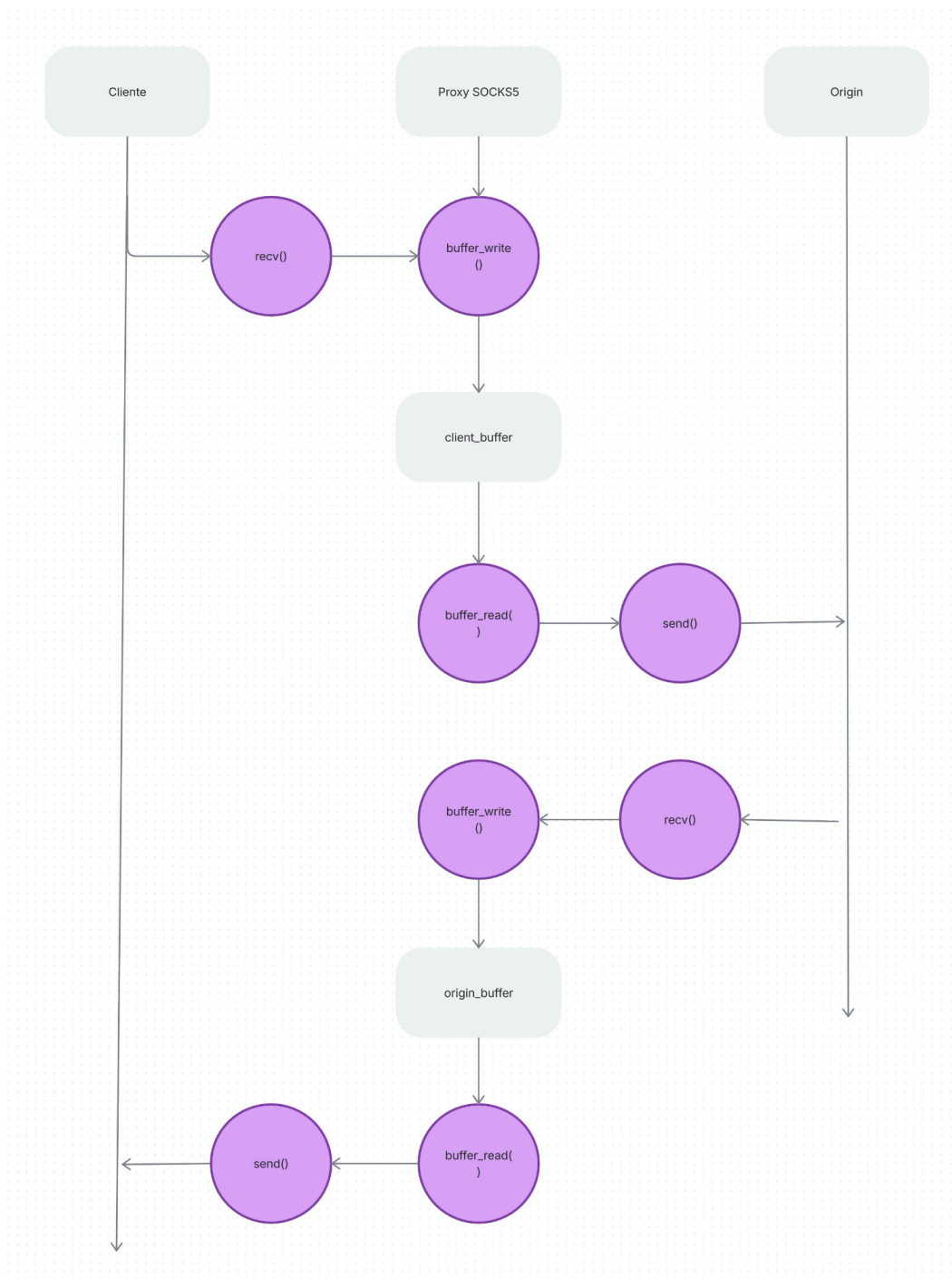


Diagrama 2: Muestra el flujo de datos de las aplicaciones y entre ellas.

- Si `'client_buffer'` tiene datos → OP_WRITE en origin.

- Si `origin\_buffer` tiene datos → OP_WRITE en cliente.
- Si los buffers no están llenos → OP_READ en ambos.

10.4.3. Thread Pool

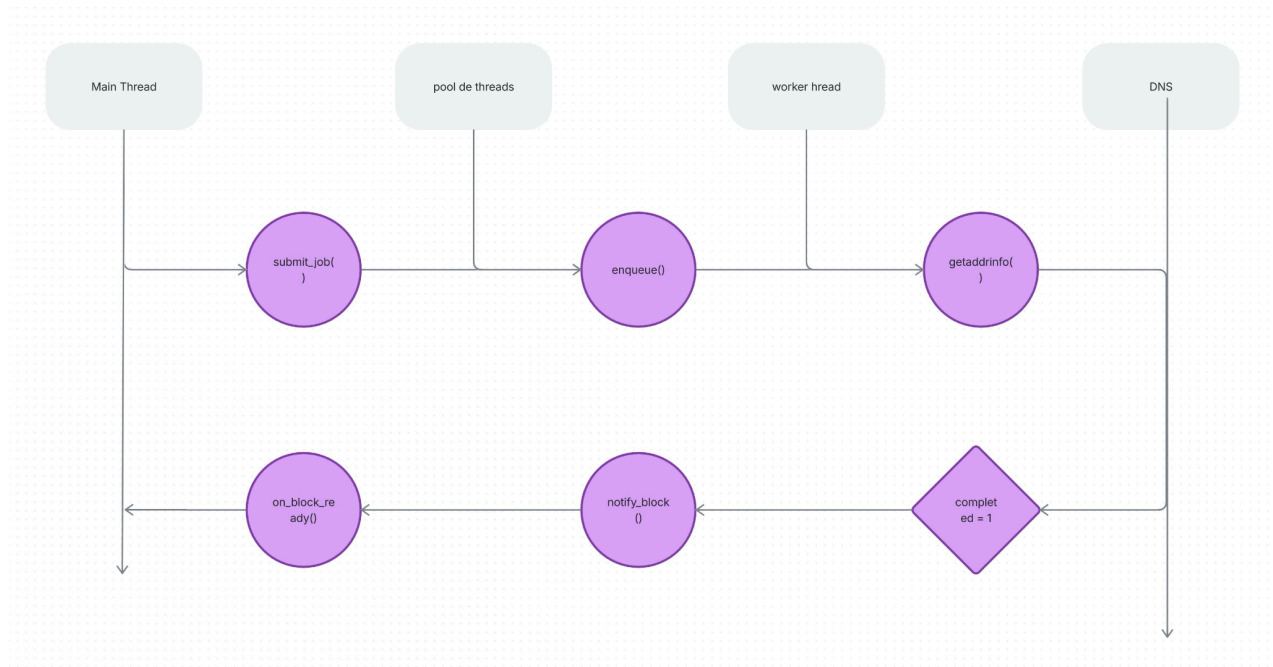


Diagrama 3: Exhibe el thread pool.

Fuentes

Butenhof, D. R. (1997). "Programming with POSIX Threads". Sección 4.2: "Work Crew".

W. Richard Stevens (2003). "UNIX Network Programming, Volume 1" -. Seccion 16

Leech, M. D. (2025). *RFC 1929: Username/Password Authentication for SOCKS V5*.

IETF Datatracker. <https://datatracker.ietf.org/doc/html/rfc1929>

Leech, M. D. (2025). *RFC 1928: SOCKS Protocol Version 5*. IETF Datatracker. <https://datatracker.ietf.org/doc/html/rfc1928>